

UNIVERSIDAD DE GUADALAJARA
Centro Universitario de Ciencias Exactas
e Ingenierías

División de Ciencias Básicas
Licenciatura en Matemáticas



Proyecto Integrador de Modelación Matemática

Análisis de conveniencia de Redes Neuronales
angostas contra profundas usando el perceptrón
para problemas de clasificación

Cristobal Cortés Gómez

Asesor: Dr. Fernando Ignacio Becerra Lopez

Guadalajara, Jal., Diciembre 2025

1 Justificación

Cuando se desea escoger la arquitectura de red para resolver un problema de clasificación, unos de los parámetros más importantes a tener en cuenta son la cantidad de neuronas y la cantidad de capas ocultas que tendrá la red neuronal. Para dicho propósito generalmente se emplean heurísticas simples, como que a más capas mejor generalizará la red Mhaskar, Liao, and Poggio 2017, mientras que con pocas capas la red será más manejable en términos de interpretabilidad de qué hace cada neurona, más estable a cambios pequeños en los pesos, así como eficiente computacionalmente.

En general se sabe que las llamadas redes angostas, es decir, con un número pequeño de capas ocultas o incluso solamente una, ya son aproximadores universales Cybenko 1989, esto usando tantas neuronas en esas capas como sean necesarias. También resulta ser que las redes profundas, es decir, usando un número limitado de neuronas por capa pero ampliando el número de capas hasta tantas como sean necesarias, son a su vez aproximadores universales Lu et al. 2017. La universalidad implica la resolución de cualquier problema de clasificación Cybenko 1989, por lo que de hecho ambas arquitecturas (angosta y profunda) son opciones viables a la hora de resolver un problema de clasificación.

El analizar lo que sucede dentro de una red neuronal es un tema de amplio estudio y que conlleva alta dificultad, ya que las mismas redes son a menudo descritas como cajas negras Nielsen 2015. Por esto, aunque los teoremas universales son antiguos, el tema de la convergencia se sigue estudiando, de modo tal que en años recientes continúan apareciendo nuevas observaciones en el comportamiento de las soluciones, como el llamado *grokking* Fan 2024, que contradice la idea establecida de que no se debe hacer sobreajuste a los datos de entrenamiento o que el sobreajuste en general es una mala señal. En los estudios se muestra que las redes profundas son más susceptibles a este fenómeno y, además, en otros se cita que pueden hacer un sobreajuste benigno Chatterji and Long 2023. Por razones como estas, el estudio del funcionamiento de las redes neuronales aún es de gran relevancia e interés.

Agregado a lo anterior, hay que tener en cuenta que otros parámetros pueden influenciar en el comportamiento de las redes y llevarnos a conclusiones distintas para la misma arquitectura, pues la función de costo o incluso el número de puntos de ejemplo pueden influir en qué arquitectura es mejor, angosta o profunda Yang and He 2024. Así mismo se entiende que el parámetro de las funciones de activación a usarse cambia ampliamente el comportamiento de las soluciones dependiendo de si la red es angosta o profunda Radhakrishnan, Belkin, and Uhler 2023.

Hay que mencionar que la discusión de cuál arquitectura es mejor en los últimos años ha apuntado hacia las redes profundas con el llamado *deep learning* Kůrková 2020; sin embargo, el debate sigue en pie, pues se ha mostrado que diversas redes profundas pueden ser implementadas en versiones angostas Ba and Caruana 2014, con lo que continúa siendo incierto el panorama de cuál arquitectura escoger.

2 Objetivos:

2.0 Objetivo Principal

Investigar la inmensa gama de posibilidades de redes entre angostas y profundas enfocándonos en la resolución de problemas de clasificación mediante un perceptrón sigmoideal para entender las diferencias entre los dos enfoques.

2.1 Objetivos específicos

- Encontrar una métrica justa para comparar redes angostas y profundas.
- Aplicar en diversas bases de datos que permitan ver de manera clara las diferencias entre las soluciones verificando lo descrito sobre la mejor generalización de las redes profundas Mhaskar, Liao, and Poggio 2017. Contrastar con escenarios reales las ventajas de usar una versión de la red angosta.
- Implementar la metodología propuesta en Ba and Caruana 2014 para transformar las redes profundas en redes angostas y verificar la eficiencia computacional.
- Estudiar la eficiencia computacional de ambos tipos de red, si es posible considerando qué tan ventajoso es una sobre otro en el hardware actual.

3 Hipótesis

Entender los dos enfoques a tal grado que podamos distinguir cuál es la arquitectura ideal para resolver un problema de clasificación real. Es decir, cuando se tenga un problema de clasificación específico, ser capaces de *a priori* seleccionar el tipo de arquitectura adecuado mediante una metodología o procedimiento bien descrito, considerando el costo computacional en todo momento.

4 Metodología

- Construir un programa que entrene un perceptrón sobre un conjunto de datos específico para resolver un problema de clasificación en el que se puedan variar el número de capas y neuronas por capa de manera sencilla.
- Encontrar un problema de clasificación con la complejidad necesaria y suficientes pero no demasiados datos para realizar pruebas.
- Desarrollar una metodología de medición que nos permita decidir cuáles redes son las que mejor resuelven nuestro problema de clasificación.

- Usar el programa creado, el problema de clasificación y la métrica con arquitecturas de red de diferentes números de capas y neuronas con el fin de realizar una estadística que muestre cómo mejoran o empeoran las soluciones conforme se agregan o quitan neuronas o capas.
- Probar la transformación de una red profunda en una angosta y si es posible viceversa.
- Probar con otros problemas de clasificación si las conclusiones sobre el primero se cumplen o si no hay consistencia en la metodología.
- Estudiar el costo computacional de las arquitecturas que mejor resuelvan el problema contra otras.
- Concluir con una receta o, en caso de que no exista, concluir qué otros métodos se pueden explorar para enfrentar el problema de decidir la mejor arquitectura entre una red angosta o profunda para un problema de clasificación específico.

5 Cronograma:

N	Actividad	Ene	Feb	Mar	Abr	May	Jun	Jul	Ago	Sep	Oct	Nov
1	Revisar estado del arte											
2	Construir programa											
3	Seleccionar problemas											
4	Buscar métrica											
5	Probar experimento											
6	Verificar costos											
7	Escribir documento											

6 Desarrollo

Un perceptrón multicapa o MLP (*Multi Layer Perceptron*) es el nombre que recibe la red neuronal artificial primitiva con la que se construyen todas las redes artificiales modernas, desde las que procesan imágenes hasta las que procesan el lenguaje natural, como se ha visto con el popular ChatGPT. Estas redes tienen una increíble gama de aplicaciones, llegando a tener utilidad en prácticamente cualquier rincón del conocimiento y la vida cotidiana.

Esta red se compone de neuronas artificiales que son modeladas por **perceptrones**, de modo tal que para una entrada dada, las neuronas la procesen y devuelvan una salida correspondiente. Las neuronas se conectan unas con otras formando una red por la que se transmite la información desde la entrada hasta la salida del modelo. La red es unidireccional (*feedforward*) y las neuronas se organizan en capas: las entradas se conectan a las neuronas de la primera capa, estas a su vez se conectan a las neuronas de la segunda, y así sucesivamente hasta que las neuronas de la última capa generan las salidas del modelo. A las capas de neuronas intermedias se les llama *capas ocultas*, mientras que a la última capa se le denomina capa de salida.

6.1 El Perceptrón y el Modelo Matemático

El **perceptrón** es el modelo más básico de una **neurona artificial** y un componente fundamental en el campo de la inteligencia artificial y el aprendizaje automático. Fue propuesto por el psicólogo y científico informático Frank Rosenblatt a finales de la década de 1950. Un perceptrón es un modelo computacional diseñado para imitar el funcionamiento básico de una neurona biológica; recibe múltiples entradas numéricas, las procesa y produce una única salida. Es la unidad fundamental a partir de la cual se construyen redes neuronales artificiales más complejas.

El funcionamiento del perceptrón se basa en varios componentes clave:

- **Entradas (x_i):** Valores numéricos que se proporcionan al modelo, provenientes de los datos o de capas anteriores.
- **Pesos (w_i):** Cada entrada tiene un peso sináptico asociado que determina su importancia relativa en la activación de la neurona. Durante el entrenamiento, estos pesos se ajustan para minimizar el error de salida.
- **Suma ponderada:** El perceptrón calcula la combinación lineal de las entradas multiplicadas por sus respectivos pesos.
- **Sesgo o Bias (b):** Un parámetro adicional que permite desplazar la función de activación, análogo al umbral de disparo en una neurona biológica.
- **Función de activación (σ):** El resultado de la suma ponderada pasa a través de una función de activación no lineal que decide la intensidad de la salida de la neurona.

Matemáticamente, la respuesta de un Perceptrón está dada por:

$$y = \sigma \left(\sum_{i=1}^n w_i x_i + b \right)$$

donde x_i representa cada una de las componentes del vector de entrada, w_i el peso sináptico asociado a dicha entrada y b el sesgo. La función σ representa la no-linealidad del sistema. Es importante destacar que la función de activación σ se aplica únicamente después de realizar la transformación lineal (suma ponderada) y suele ser la misma para todas las neuronas de una misma capa, aunque puede variar entre capas distintas.

En cuanto a la red neuronal MLP, su modelación se sigue de conectar la respuesta de cada perceptrón en una capa a la entrada de cada perceptrón en la siguiente, hasta completar la red entera, considerando que los primeros perceptrones están conectados a las entradas directas del modelo. El ejemplo más sencillo de una red multicapa sería con un solo perceptrón/neurona por capa. Su respuesta, para una red de n capas, estaría dada por la composición de funciones:

$$y = \sigma^{(n)} \left(w^{(n)} \dots \sigma^{(2)} \left(w^{(2)} \sigma^{(1)} \left(w^{(1)} x + b^{(1)} \right) + b^{(2)} \right) \dots + b^{(n)} \right)$$

Para generalizar esto a múltiples neuronas por capa, empleamos notación matricial. Si definimos $\mathbf{W}^{(l)}$ como la matriz de pesos de la capa l , $\mathbf{b}^{(l)}$ como el vector de sesgos y $\mathbf{a}^{(l)}$ como el vector de

activaciones (salidas) de la capa l , la propagación hacia adelante se define recursivamente como:

$$\mathbf{a}^{(l)} = \sigma(\mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$$

Donde $\mathbf{a}^{(0)} = \mathbf{X}$ (entrada). Así, la red se conforma por la respuesta de múltiples transformaciones afines seguidas de no-linealidades puntuales. De modo tal que, en cierto sentido, el MLP extiende a los modelos de Regresión Lineal y, más específicamente, al modelo de Regresión Logística, potenciándolos gracias a su capacidad de representar funciones no lineales complejas mediante la composición.

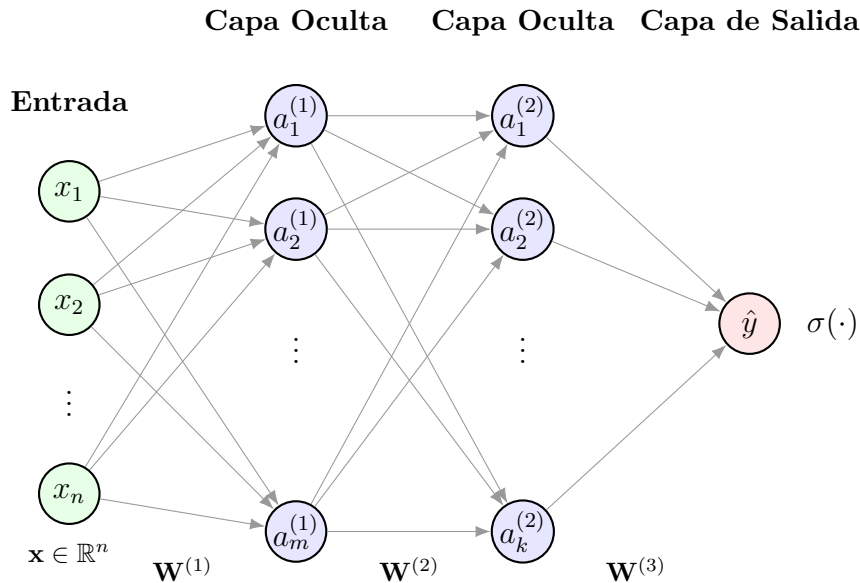


Figure 1: Diagrama esquemático de un Perceptrón Multicapa (MLP) profundo. La información fluye desde la capa de entrada (izquierda) a través de capas ocultas sucesivas hasta la capa de salida (derecha). Cada conexión representa un peso sináptico w_{ji} .

Para simplificar el análisis y la notación en problemas de optimización, es común agrupar todos los parámetros ajustables del modelo (pesos y sesgos de todas las capas) en un único vector de parámetros denotado como θ . Así, podemos referirnos a la red neuronal completa como una función parametrizada $f(x; \theta)$.

El modelo de nuestra red al final es una *función* $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$. El comportamiento de esta función está completamente determinado por los valores en θ . Además, dada la naturaleza de las transformaciones lineales junto con el hecho de que las funciones σ (como la sigmoide) son continuas, resulta ser que el modelo MLP es, en su totalidad, una función continua.

6.2 Universalidad y Análisis Funcional

En términos de análisis matemático, hay muchas preguntas que nos podemos hacer sobre la función que representa la red. La más importante es si existe una correspondencia completa: ya sabemos que la función que obtenemos es continua, pero ¿será posible que cualquier función continua que deseemos aproximar pueda ser representada por una red neuronal?

Esta pregunta motivó muchos estudios en la segunda mitad del siglo XX. Fue finalmente demostrada de manera formal por Cybenko en 1989, quien utilizó el análisis funcional para dicho propósito y enfatizó que esta área de las matemáticas podía ser usada para abordar la teoría de redes neuronales. La demostración extiende este resultado a funciones más allá de las continuas bajo ciertas condiciones de medida.

Formalmente, el Teorema de Aproximación Universal de Cybenko establece que dada una red neuronal con una sola *capa oculta* de neuronas representadas por perceptrones con funciones de activación *sigmoidales*, esta puede aproximar cualquier función continua f en un conjunto compacto de $\mathbb{R}^n$. Esto significa que las redes neuronales de una sola capa son densas en el espacio de las funciones continuas $C(K)$. La desigualdad fundamental se expresa como:

$$|g(x) - f(x)| < \epsilon \quad \forall x \in K$$

Donde $g(x)$ representa la función salida de la red neuronal (la suma finita de la forma $\sum \alpha_j \sigma(y_j^T x + \theta_j)$) y ϵ es un error arbitrariamente pequeño. Esto garantiza la existencia de una red que aproxima la función objetivo, aunque no nos dice cómo encontrar los pesos.

La demostración continúa con resultados que muestran la densidad en espacios de funciones integrables (L^2). A estos teoremas se les conoce popularmente como Teoremas de Aproximación Universal, y un corolario de ellos es que se puede resolver cualquier *problema de clasificación*. Esto es, dado un conjunto de datos pertenecientes cada uno a una clase, es posible encontrar una función que relacione uno a uno cada dato con su respectiva clase (por ejemplo, clasificar imágenes de animales).

Cabe destacar que, en el contexto del aprendizaje estadístico, es convención denotar al conjunto total de parámetros ajustables del modelo (pesos w y sesgos b) mediante el vector θ . Esta notación resalta la interpretación del modelo como una aproximación probabilística, $P(y|x; \theta)$, especialmente útil en escenarios donde se desconoce la función generadora de datos subyacente y se trabaja con muestras que contienen ruido inherente. En el presente trabajo adoptaremos esa notación para hablar de los parámetros del modelo a conveniencia cuando no requiramos mencionar explícitamente los pesos W y sesgos b .

Para esta tesis nos enfocaremos en los *problemas de clasificación binaria*, donde el conjunto de datos se relaciona solo con dos clases: 0 y 1 (colores claros y oscuros, diagnósticos positivos y negativos, etc.).

Históricamente, el hecho de que las redes con una y dos capas tuvieran resultados teóricos demostrando capacidades tan fuertes, hizo que temporalmente se perdiera el interés en explorar redes con mayor número de capas, bajo la premisa de "¿para qué más si ya sobra?". Sin embargo, el auge de mejores tecnologías de procesamiento, concretamente las GPU, devolvió el interés en estas redes al mostrarse superiores empíricamente para resolver problemas con conjuntos muy grandes de datos, dando nacimiento al *Deep Learning*. Más tarde, aparecieron teoremas complementarios Lu et al. 2017 que demostraban la capacidad de aproximación universal desde la perspectiva de la profundidad (ancho acotado, profundidad variable).

Muchos teoremas modernos como el mencionado Lu et al. 2017 no usan funciones de activación sigmoidales, basándose en funciones como ReLU (*Rectified Linear Unit*), que describe una función rampa ($f(x) = \max(0, x)$). No obstante, la función que usamos para este documento es la función

logística (sigmoide), la cual es diferenciable en todo su dominio, con rango $(0, 1)$, estrictamente creciente y antisimétrica respecto a su punto medio, modelando la probabilidad de activación neuronal clásica en donde las neuronas responden activándose 1 y desactivándose 0.

6.3 El Problema de la Optimización y el Aprendizaje

La motivación de esta tesis radica en la competencia entre redes angostas (que aproximan agregando neuronas a una capa fija) y redes profundas (que aproximan agregando capas con ancho fijo). Si bien la teoría garantiza la *existencia* de una red para ambos casos, en la práctica nos enfrentamos al problema de encontrar los valores de los pesos θ .

Aquí surge una distinción crucial: la demostración de Cybenko es existencial, no constructiva. Nos dice que "existe" una configuración de pesos que resuelve el problema, pero encontrarla es un desafío de optimización. Para ello, disponemos de algoritmos que buscan minimizar una **función de costo** (o función de pérdida), la cual mide la discrepancia entre la predicción de la red y el valor real esperado.

El método estándar para encontrar estos pesos es el *descenso por gradiente*. Dado que nuestra red es una función diferenciable (gracias al uso de sigmoides), podemos calcular el gradiente de la función de costo respecto a todos los parámetros, $\nabla_{\theta} J(\theta)$. Este vector nos indica la dirección de máximo crecimiento del error, por lo que actualizamos los pesos en la dirección opuesta:

$$\theta_{\text{nuevo}} = \theta_{\text{viejo}} - \eta \nabla_{\theta} J(\theta)$$

Donde η es la tasa de aprendizaje.

Sin embargo, surgen problemas paralelos al intentar encontrar las soluciones. Uno es el sobreajuste (*overfitting*), donde la red aprende el ruido de los datos de entrenamiento en lugar de la función subyacente. Otro es la dificultad del paisaje de optimización: aunque una red tenga la capacidad teórica de representar la función, el algoritmo de descenso por gradiente puede quedar atrapado en mínimos locales subóptimos o zonas de gradiente nulo, impidiendo encontrar los parámetros ideales.

Durante la investigación bibliográfica se han identificado fórmulas teóricas que permiten acotar el número mínimo de neuronas necesarias para que una arquitectura dada sea capaz de aproximar la función objetivo. No obstante, la obtención práctica de la solución se enfrenta a dos barreras fundamentales que deben diferenciarse claramente en nuestro análisis.

El **primer problema es de índole estadística y de generalización**. Dado que en situaciones reales solo disponemos de un conjunto finito de ejemplos, a menudo contaminados por ruido, el algoritmo de aprendizaje tiende a encontrar un conjunto de parámetros empíricos $\hat{\theta}$ que minimiza el error sobre la muestra disponible (interpolación), pero que no necesariamente corresponde a los parámetros ideales θ^* que minimizan el error sobre la distribución real de los datos (extrapolación). Es decir, existe el riesgo de que la red aprenda a describir las particularidades y el ruido del subconjunto de datos, en lugar de capturar la función generadora subyacente.

El **segundo problema es de optimización y alcanzabilidad**. Aún en el supuesto de que la brecha entre $\hat{\theta}$ y θ^* fuera despreciable, no existe garantía de que los métodos de optimización actuales, como el descenso por gradiente, sean capaces de navegar el paisaje de la función de costo hasta

encontrar dicha solución global. Esto se evidencia notablemente en la literatura sobre *compresión de modelos* Ba and Caruana 2014: se ha demostrado que es posible "destilar" el conocimiento de una red profunda compleja en una red angosta, lo cual prueba matemáticamente que la red angosta posee la *capacidad teórica* de representar la solución. Sin embargo, resulta extremadamente difícil, si no imposible, entrenar esa misma red angosta desde cero para llegar a ese nivel de rendimiento. Esto sugiere que la limitante no reside en la capacidad de representación de la arquitectura, sino en la ineficacia del algoritmo de búsqueda para encontrar los mínimos en arquitecturas angostas sin una guía externa.

En conclusión, consideramos indispensable abordar ambos desafíos simultáneamente: no basta con verificar que una red tenga la capacidad teórica de resolver el problema, sino que es necesario validar si es factible alcanzar dicha solución con los algoritmos de optimización disponibles. A diferencia de estudios previos que suelen aislar estas variables, este trabajo busca contrastar ambas limitaciones para determinar la arquitectura más apropiada para resolver el problema de clasificación.

6.4 El problema de clasificación binaria

Cuando hablamos de un problema de clasificación binaria, nos referimos a uno con condiciones específicas. En este tipo de problemas se cuenta con un conjunto de datos de entrenamiento en el que cada ejemplo se compone de un par $(\mathbf{x}, y)$, con $\mathbf{x} \in \mathbb{R}^n$ correspondiente a la entrada que debe ser evaluada por la red, y $y \in \mathbb{R}$ (típicamente $y \in \{0, 1\}$) correspondiente a la salida esperada o clase objetivo.

Estas componentes para todo el conjunto de datos se pueden representar mediante una matriz de entradas $\mathbf{X}$ y un vector de salidas $\mathbf{Y}$. Si disponemos de k datos de ejemplo, organizamos la matriz $\mathbf{X}$ de dimensiones $(n \times k)$ acomodando cada vector de entrada $\mathbf{x}$ como una columna, mientras que las respuestas conforman un vector fila $\mathbf{Y}$ de dimensión $(1 \times k)$, respetando el orden de las columnas en $\mathbf{X}$.

Esto implica que si f es la función ideal que resuelve el problema de clasificación tal que $f(\mathbf{x}) = y$, entonces para el conjunto completo se cumple:

$$F(\mathbf{X}) = \mathbf{Y}$$

donde $F(\mathbf{X}) = (f(\mathbf{x}_1), f(\mathbf{x}_2), \dots, f(\mathbf{x}_k))$.

Así, el problema de optimización que queremos resolver al entrenar la red neuronal es encontrar la función parametrizada G_θ que mejor aproxime a f , minimizando una función de costo. Utilizando el Error Cuadrático Medio, buscamos:

$$\theta^* = \underset{\theta}{\operatorname{argmin}} (||G_\theta(\mathbf{X}) - \mathbf{Y}||^2)$$

Aquí asumimos que G_θ es la construcción de nuestra red neuronal que intenta aproximarse a f , de modo que tras el entrenamiento se cumpla $G_\theta(\mathbf{x}) \approx y$.

Es fundamental hacer hincapié en que el hecho de que la red resuelva adecuadamente para este conjunto limitado de ejemplos no implica que la función se extrapole correctamente a ejemplos con

los que la red no ha sido entrenada. Más aún, debido al ruido natural presente en la recolección de datos, intentar reducir la función de costo a un cero absoluto suele ser contraproducente, pues la red aprendería a predecir el ruido inherente, empeorando su capacidad de generalización.

Con esto en mente, el proceso para encontrar los pesos y sesgos que mejor aproximen la función f se realiza mediante el algoritmo de **descenso por gradiente**. La regla de actualización iterativa es:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \|G_{\theta}(\mathbf{X}) - \mathbf{Y}\|^2$$

donde η es la tasa de aprendizaje y θ_0 arbitrario, generalmente aleatorio. El vector de parámetros θ se expresa explícitamente como la concatenación de todos los pesos $\mathbf{w}$ y sesgos $\mathbf{b}$ de la red:

$$\begin{aligned}\theta &= [\mathbf{w}, \mathbf{b}] \\ &= (w_{11}^{(1)}, \dots, w_{nm}^{(L)}, b_1^{(1)}, \dots, b_m^{(L)})\end{aligned}$$

siendo L el número total de capas (incluyendo la de salida). La función de la red $G_{\theta}(\mathbf{X})$ se define recursivamente como una composición de transformaciones lineales y activaciones:

$$G_{\theta}(\mathbf{X}) = \sigma^{(L)} (\mathbf{W}^{(L)} \dots \sigma^{(1)} (\mathbf{W}^{(1)} \mathbf{X} + \mathbf{B}^{(1)}) \dots + \mathbf{B}^{(L)})$$

El cálculo del gradiente ∇_{θ} necesario para la actualización se obtiene mediante la regla de la cadena (algoritmo de *Backpropagation*), resultando en un vector que contiene las derivadas parciales de la función de costo respecto a cada componente de θ :

$$\nabla_{\theta} J = \left(\frac{\partial J}{\partial w_{11}^{(1)}}, \dots, \frac{\partial J}{\partial w_{nm}^{(L)}}, \frac{\partial J}{\partial b_1^{(1)}}, \dots, \frac{\partial J}{\partial b_m^{(L)}} \right)$$

Una vez definido el mecanismo matemático para ajustar los parámetros y minimizar el error teórico sobre los datos conocidos, surge la necesidad de interpretar la salida de la red para tomar decisiones y evaluar qué tan bien generaliza el modelo ante datos nuevos.

Finalmente, para operativizar la evaluación de las arquitecturas en el contexto de clasificación binaria, es necesario traducir la salida probabilística de la red en una decisión categórica concreta. Dado que nuestra función de salida es sigmoideal, la red entrega un valor continuo $\hat{y} \in (0, 1)$ que representa la probabilidad condicional $P(y = 1 | \mathbf{x}; \theta)$. Para asignar una etiqueta de clase definitiva $y_{pred} \in \{0, 1\}$, establecemos un umbral de decisión (típicamente $\tau = 0.5$), definido por la regla:

$$y_{pred} = \begin{cases} 1 & \text{si } \hat{y} \geq \tau \\ 0 & \text{si } \hat{y} < \tau \end{cases}$$

La validación de la arquitectura no se realiza sobre la totalidad de los datos disponibles de manera uniforme. Para medir la capacidad de generalización —que es el criterio central para comparar redes profundas contra angostas—, el conjunto de datos se particiona en dos subconjuntos disjuntos: un conjunto de *entrenamiento*, utilizado exclusivamente para el ajuste iterativo de los parámetros θ mediante el descenso por gradiente, y un conjunto de *prueba* (o validación), que permanece "invisible" para la red durante la fase de aprendizaje y se utiliza únicamente para la evaluación final.

El análisis crítico se basa en contrastar las métricas de desempeño obtenidas en estos dos conjuntos. Es en la diferencia o "brecha" entre ambos resultados donde se diagnostica el comportamiento estructural del modelo:

- Un alto rendimiento en el conjunto de entrenamiento acompañado de un rendimiento degradado en el de prueba indica **sobreajuste** (*overfitting*). Esto implica que la red ha "memorizado" el ruido o las particularidades específicas de los datos de entrenamiento, perdiendo la capacidad de extrapolar a ejemplos nuevos.
- Un rendimiento deficiente en ambos conjuntos sugiere **subajuste** (*underfitting*), lo cual señala que la arquitectura carece de la capacidad de representación necesaria para modelar la complejidad del problema, o que el algoritmo de optimización no ha logrado converger.

Por lo tanto, la arquitectura óptima no es necesariamente la que minimiza el error de entrenamiento a cero, sino aquella que logra el menor error de prueba posible, manteniendo una brecha de generalización mínima. Dado que la simple tasa de aciertos (*accuracy*) puede resultar engañosa, especialmente en bases de datos donde una clase es dominante, en las secciones siguientes definiremos métricas más robustas (Precisión, Recuperación y F1-Score) que servirán como el estándar cuantitativo para nuestra metodología de comparación.

6.5 Parte computacional

Para llevar a la práctica las fórmulas del gradiente y obtener el vector $\nabla_{\theta} J$ de manera eficiente, utilizamos el algoritmo de **Retropropagación** (*Backpropagation*). Este método computa las derivadas parciales de la función de costo aplicando la regla de la cadena de forma recursiva, comenzando desde la capa de salida y avanzando hacia atrás hasta la primera capa.

El algoritmo se basa fundamentalmente en calcular el término de error $\delta^{(l)}$ para la capa l utilizando el error ya calculado de la capa siguiente ($l + 1$):

$$\delta^{(l)} = ((\mathbf{W}^{(l+1)})^T \delta^{(l+1)}) \odot \sigma'(\mathbf{z}^{(l)})$$

donde $\odot$ representa la multiplicación elemento a elemento (producto de Hadamard) y $\mathbf{z}^{(l)}$ es la entrada ponderada de la neurona. Con estos errores, las derivadas finales para los pesos se obtienen mediante productos matriciales.

Al analizar estas operaciones, nos damos cuenta de que el esfuerzo computacional es considerable y depende directamente del tamaño del problema. En términos de complejidad asintótica, el costo de realizar un paso de entrenamiento completo llamado una época, es decir, calcular θ_{i+1} escala según $O(k \cdot w)$, siendo k el número de ejemplos de entrenamiento y w el número total de parámetros (pesos y sesgos) en la red Goodfellow, Bengio, and Courville 2016.

Por esta razón, decidimos incluir el costo computacional como un punto clave en nuestra comparativa. Nos interesa contrastar que, aunque dos redes puedan tener la misma cantidad total de parámetros w (y por ende la misma complejidad teórica), su ejecución en hardware real difiere: las redes **angostas** realizan pocas operaciones con matrices muy grandes —lo cual es altamente

paralelizable en GPUs—, mientras que las redes **profundas** implican una secuencia larga de operaciones con matrices más pequeñas, donde cada capa debe esperar a la anterior, introduciendo latencia secuencial.

7 Métodos para la resolución del análisis

7.1 Programación de la red

7.1.1 Un programa versátil como base experimental

Para llevar a cabo este estudio se siguió la metodología estipulada implementando una herramienta de software versátil que permite crear redes neuronales, almacenarlas en memoria, evaluarlas y optimizarlas mediante descenso por gradiente utilizando el algoritmo de *Backpropagation*.

Este programa se diseñó con flexibilidad en mente: todos los *hiperparámetros* de la red son fácilmente configurables. Además, implementa herramientas adicionales para potenciar el desempeño del descenso por gradiente, tales como variantes estocásticas (SGD), tasas de aprendizaje adaptativas, condiciones de parada personalizables, y la posibilidad de intercambiar funciones de costo y de activación. Incluso cuenta con un módulo de verificación de gradiente numérico para asegurar que el complejo cálculo analítico de las derivadas se esté ejecutando correctamente.

La entrada principal del programa es la definición de la arquitectura de la red, la cual se especifica mediante una tupla o lista ordenada de enteros. Esta lista representa el número de neuronas en cada capa, incluyendo explícitamente la capa de entrada y la de salida. Por ejemplo, la tupla (5, 3, 10, 1) define una red con 5 entradas, una primera capa oculta de 3 neuronas, una segunda capa oculta de 10 neuronas y una única salida.

Una vez definida la arquitectura, el entrenamiento se realiza suministrando los datos correspondientes en forma de pares de matrices $\mathbf{X}$ y $\mathbf{Y}$ compuestas por números de punto flotante.

7.1.2 Un segundo programa más potente pero limitado

Debido a que los problemas de clasificación seleccionados tienen pocos miles de ejemplos de media y a que los experimentos se terminaron corriendo por hasta un millón de épocas, se realizó un segundo programa que procesa los datos en ultra paralelo usando la GPU para ahorrar tiempo durante las pruebas.

Para este segundo programa solo se priorizó su eficiencia computacional y es más limitado pero tiene lo necesario para ejecutar el descenso por gradiente y calcular el F1Score.

7.2 Selección de problemas de clasificación

Se determinó que los problemas de clasificación a utilizar no deberían requerir un preprocesamiento complejo; los datos debían estar listos para ser introducidos a la red desde su descarga. Esta decisión surgió tras una experiencia inicial con la base de datos de sobrevivientes del Titanic, la cual requirió un tratamiento extensivo que introdujo incertidumbre sobre si los fallos en la resolución se debían a la arquitectura de la red o a la calidad del preprocesamiento.

Posteriormente, se concluyó que para realizar una mejor comparativa entre las diferentes arquitecturas era necesario medir el desempeño en una variedad de contextos. Por ello, se seleccionó una colección inicial de 7 problemas de clasificación, a los cuales se añadieron posteriormente otros 7 de dificultad media, conformando un banco de pruebas diverso.

7.3 Metodología de comparación

Al momento de plantear la comparación, nos enfrentamos a la inmensa cantidad de combinaciones posibles de neuronas y capas. Dado que es inviable probarlas todas, el primer desafío fue definir qué redes específicas debíamos contrastar para asegurar un análisis justo. Se estableció que la comparación debía ser 1 a 1 entre redes angostas y profundas, normalizando el **costo computacional** como criterio de equivalencia.

Nuestra premisa fundamental fue que el indicador más fiable del costo computacional es la cantidad total de pesos (parámetros) de la red, ya que este número es directamente proporcional a la cantidad de operaciones de multiplicación y suma requeridas para evaluar el modelo.

Para simplificar el análisis, impusimos una restricción estructural: las redes profundas tendrían un número constante de neuronas por capa oculta, mientras que las redes angostas consistirían en una única capa oculta con un número variable de neuronas.

El problema se formuló matemáticamente de la siguiente manera:

Consideremos un par de redes con dimensión de entrada N y dimensión de salida M .

- Una red **angosta** con una sola capa oculta de L neuronas.
- Una red **profunda** con K capas ocultas, cada una con $\hat{L}$ neuronas.

El número total de pesos $\#w$ para cada arquitectura está dado por:

$$\#w = \begin{cases} NL + LM & \text{Para la red angosta} \\ N\hat{L} + (K - 1)\hat{L}^2 + \hat{L}M & \text{Para la red profunda} \end{cases}$$

Nuestro objetivo es igualar la cantidad de parámetros de ambas arquitecturas para hacerlas computacionalmente equivalentes. Igualando las expresiones y despejando L , obtenemos la relación

necesaria para la red angosta:

$$L = \frac{N\hat{L} + (K-1)\hat{L}^2 + \hat{L}M}{N+M}$$

A modo de ejemplo, si tenemos $N = 5$, $M = 1$, $\hat{L} = 3$ y $K = 3$:

$$L = \frac{15 + 18 + 3}{5 + 1} = \frac{36}{6} = 6$$

Esto significa que una red profunda de 3 capas ocultas con 3 neuronas cada una es equivalente en número de parámetros a una red angosta con una única capa oculta de 6 neuronas.

Ahora bien, surge la pregunta: ¿cuál arquitectura utiliza más neuronas en total? El número total de neuronas en la red profunda es $K\hat{L}$, mientras que en la angosta es simplemente L .

Para averiguarlo, planteamos la desigualdad asumiendo que la profunda tiene más o igual neuronas ($\hat{L}K \geq L$) y vemos bajo qué condiciones se cumple:

$$\begin{aligned}\hat{L}K &\geq L \\ \hat{L}K &\geq \frac{N\hat{L} + (K-1)\hat{L}^2 + \hat{L}M}{N+M}\end{aligned}$$

Dividiendo toda la expresión por $\hat{L}$ (que es positivo):

$$\begin{aligned}K &\geq \frac{N + K\hat{L} - \hat{L} + M}{N+M} \\ K(N+M) &\geq N + M - \hat{L} + K\hat{L} \\ K(N+M) - K\hat{L} &\geq N + M - \hat{L} \\ K(N+M - \hat{L}) &\geq N + M - \hat{L}\end{aligned}$$

Llegados a este punto, para despejar K debemos dividir por el término $(N + M - \hat{L})$. Dado que en una desigualdad al dividir por un número negativo el signo se invierte, tenemos dos casos posibles para el resultado de K :

$$K \begin{cases} \geq 1 & \text{si } \hat{L} < N + M \quad (\text{el término es positivo}) \\ \leq 1 & \text{si } \hat{L} > N + M \quad (\text{el término es negativo}) \end{cases}$$

Por lo que la respuesta a si la red profunda tendrá más neuronas en total que la angosta depende exclusivamente de si el número de neuronas por capa oculta $\hat{L}$ es menor o mayor que la suma de las dimensiones de entrada y salida $N + M$.

- Si es menor ($\hat{L} < N + M$), la condición se cumple siempre, pues por definición $K \geq 1$. En este caso, la red profunda tiene **más** neuronas.

- Si es mayor ($\hat{L} > N + M$), la condición $K \leq 1$ solo se cumpliría si $K = 1$. Para cualquier red realmente profunda ($K > 1$), la desigualdad no se cumple, lo que significa que la red angosta tiene **más** neuronas.

Concretamente, para redes con el mismo número de parámetros:

$$\begin{array}{ll} \text{Neuronas Profunda} > \text{Neuronas Angosta} & \text{si } \hat{L} < N + M \\ \text{Neuronas Profunda} < \text{Neuronas Angosta} & \text{si } \hat{L} > N + M \\ \text{Neuronas Profunda} = \text{Neuronas Angosta} & \text{si } \hat{L} = N + M \end{array}$$

Comprobamos esta última igualdad sustituyendo en la ecuación original de L :

$$L = \frac{N\hat{L} + (K-1)\hat{L}^2 + \hat{L}M}{N+M} = \frac{\hat{L}(N+M) + (K-1)\hat{L}^2}{N+M}$$

Si $\hat{L} = N + M$, entonces:

$$L = \frac{\hat{L}^2 + (K-1)\hat{L}^2}{\hat{L}} = \hat{L} + (K-1)\hat{L} = K\hat{L}$$

Lo cual confirma que si $\hat{L} = N + M$, ambas redes tienen exactamente la misma cantidad de neuronas y de pesos.

Esta regla nos proporciona una estrategia clara para el diseño experimental. Para este proyecto, decidimos **comparar redes que tengan tanto la misma cantidad de pesos como la misma cantidad de neuronas**. Esto se logra definiendo la arquitectura profunda de tal manera que el ancho de sus capas ocultas sea igual a la suma de las dimensiones de entrada y salida: $\hat{L} = N + M$. Dado que trabajamos con problemas de clasificación binaria, la salida es siempre $M = 1$, por lo que fijamos $\hat{L} = N + 1$.

En resumen, el experimento consiste en tomar un problema con dimensión de entrada N , construir redes profundas con ancho constante de $N + 1$ neuronas, y compararlas contra redes angostas equivalentes que tengan $L = K(N + 1)$ neuronas en su única capa oculta.

7.3.1 Comparación 1 a 1: Angosta vs. Profunda

Para comparar los resultados de dos arquitecturas, primero entrenamos ambas redes. Dado que el entrenamiento es un proceso estocástico (dependiente de la inicialización aleatoria y el orden de los datos), es necesario repetirlo múltiples veces para obtener resultados estadísticamente significativos. Se realizan al menos 5 ejecuciones de entrenamiento para cada arquitectura sobre el problema dado.

La métrica principal escogida es el *F1-Score*. Esta métrica es más robusta que la simple tasa de aciertos, ya que penaliza los desequilibrios entre la precisión (positivos predichos correctamente) y la recuperación (positivos reales detectados). Su rango es $[0, 1]$, donde 1 indica un desempeño perfecto.

Los criterios de evaluación son los siguientes:

- **Rendimiento Absoluto:** Se observa el *F1-Score* tanto en el conjunto de entrenamiento como en el de prueba. Buscamos valores cercanos a 1 en ambos casos.
- **Brecha de Generalización:** Se analiza la diferencia entre la puntuación de entrenamiento y la de prueba. Una diferencia amplia es un indicador claro de *sobreajuste* (overfitting), lo cual se considera un resultado negativo.
- **Estabilidad y Convergencia:** Se toma en cuenta la cantidad de iteraciones (épocas) necesarias para alcanzar la convergencia. La varianza en este parámetro entre las distintas ejecuciones nos habla de la robustez de la arquitectura para resolver el problema de manera consistente.

Cabe destacar que la evaluación en estos tres aspectos puede ser independiente; podría darse el caso de que una arquitectura sea preferible por su estabilidad o capacidad de generalización, aunque su puntuación absoluta sea ligeramente inferior.

7.3.2 Calificación general

Dado que para cada problema de clasificación se probarán distintas configuraciones (variando la profundidad K y el ancho correspondiente L), analizaremos el progreso del desempeño conforme aumenta la complejidad de la red. Lo teóricamente esperado es que, al alcanzar cierta cantidad de neuronas, el rendimiento se estabilice, indicando que la red ha alcanzado la *capacidad de representación* necesaria para aproximar la función subyacente del problema.

8 Marco Teórico

A continuación, se presentan las definiciones formales y la notación matemática que sustentan el desarrollo del proyecto.

Definición 1 (Modelo de Red Neuronal MLP): Un Perceptrón Multicapa (MLP) es una función $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, compuesta por una secuencia de L transformaciones afines seguidas de activaciones no lineales elemento a elemento. Para una entrada $\mathbf{x} \in \mathbb{R}^n$, la red se define recursivamente como:

$$\begin{aligned}\mathbf{a}^{(0)} &= \mathbf{x} \\ \mathbf{z}^{(l)} &= \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \\ \mathbf{a}^{(l)} &= \sigma(\mathbf{z}^{(l)}) \quad \text{para } l = 1, \dots, L\end{aligned}$$

Donde $\mathbf{W}^{(l)}$ es la matriz de pesos, $\mathbf{b}^{(l)}$ el vector de sesgo y σ la función de activación. La salida de la red es $\mathbf{a}^{(L)}$.

Definición 2 (Función Sigmoide Logística): La función logística $\sigma : \mathbb{R} \rightarrow (0, 1)$ está definida por:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Esta función es monótona creciente, acotada y diferenciable, con derivada $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

Definición 3 (Gradiente): Sea $f : \mathbb{R}^n \rightarrow \mathbb{R}$ una función diferenciable. El gradiente de f en $\mathbf{x}$, denotado como $\nabla f(\mathbf{x})$, es el vector de campo que contiene las derivadas parciales de f :

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right]$$

Definición 4 (Mínimos Locales y Globales): Dado un funcional de costo $J(\theta)$, un punto θ^* es un mínimo global si $J(\theta^*) \leq J(\theta)$ para todo θ en el dominio. Es un mínimo local si existe una vecindad V alrededor de θ^* tal que $J(\theta^*) \leq J(\theta)$ para todo $\theta \in V$. El entrenamiento de redes neuronales busca aproximar el mínimo global, aunque a menudo converge a mínimos locales.

Definición 5 Definimos θ como el vector que concatena la totalidad de los parámetros ajustables del modelo (pesos y sesgos) de las L capas de la red:

$$\theta = \{\mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \dots, \mathbf{W}^{(L)}, \mathbf{b}^{(L)}\} \in \mathbb{R}^d$$

donde d representa la dimensión total del espacio de parámetros. Bajo una interpretación probabilística, la red neuronal no determina una salida determinista absoluta, sino que modela una distribución de probabilidad condicional $P(y|\mathbf{x}; \theta)$.

Definición 6 (Descenso por Gradiente): Es un algoritmo de optimización iterativo de primer orden para encontrar un mínimo local de una función diferenciable. La regla de actualización es:

$$\theta_{t+1} = \theta_t - \theta \nabla J(\theta_t)$$

donde $\theta > 0$ es la tasa de aprendizaje.

Definición 7 (Función de Costo Cuadrática): Para un problema de regresión o clasificación (donde la salida es tratada como probabilidad continua), el Error Cuadrático Medio (MSE) se define como:

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \|\mathbf{y}^{(i)} - \hat{\mathbf{y}}^{(i)}(\theta)\|^2$$

donde $\mathbf{y}^{(i)}$ es la etiqueta real y $\hat{\mathbf{y}}^{(i)}$ la predicción de la red para el ejemplo i .

Cálculo del Gradiente (Backpropagation): El gradiente de la función de costo respecto a los pesos de la red se calcula mediante la regla de la cadena, propagando el error desde la salida hacia atrás:

$$\frac{\partial J}{\partial w_{jk}^{(l)}} = \frac{\partial J}{\partial z_j^{(l)}} \cdot \frac{\partial z_j^{(l)}}{\partial w_{jk}^{(l)}} = \delta_j^{(l)} a_k^{(l-1)}$$

Donde $\delta_j^{(l)}$ representa el término de error en la neurona j de la capa l .

Definición 8 (Métricas de Clasificación): Para evaluar el desempeño en clasificación binaria, se utilizan las siguientes métricas basadas en la matriz de confusión (Verdaderos Positivos VP , Falsos Positivos FP , Verdaderos Negativos VN , Falsos Negativos FN):

- **Precisión (Precision):** Proporción de predicciones positivas correctas.

$$P = \frac{VP}{VP + FP}$$

- **Recuperación (Recall):** Proporción de casos positivos reales detectados correctamente.

$$R = \frac{VP}{VP + FN}$$

- **F1-Score:** La media armónica entre precisión y recuperación, útil para clases desbalanceadas.

$$F1 = 2 \cdot \frac{P \cdot R}{P + R}$$

Definición 8 (Producto de Hadamard): Es una operación binaria que toma dos matrices de las mismas dimensiones y produce otra matriz donde cada elemento ij es el producto de los elementos ij de las matrices originales. Se denota comúnmente con el símbolo $\odot$. Si $\mathbf{A}$ y $\mathbf{B}$ son matrices de dimensión $m \times n$, entonces:

$$(\mathbf{A} \odot \mathbf{B})_{ij} = A_{ij} \cdot B_{ij}$$

En redes neuronales, esta operación es fundamental para aplicar la derivada de la función de activación a cada neurona de una capa simultáneamente durante la retropropagación.

Definición 9 (Algoritmo de Retropropagación): Conocido en inglés como *Backpropagation*, es el método estándar para calcular el gradiente de la función de costo $\nabla_{\theta} J$ en redes multicapa. Se basa en la aplicación recursiva de la regla de la cadena desde la capa de salida hacia la entrada, permitiendo calcular cómo afecta cada peso individual al error total sin necesidad de evaluar la red completa para cada parámetro por separado.

Definición 10 (Notación O Grande y Complejidad): La notación $O(g(n))$ se utiliza para describir el comportamiento límite de una función cuando su argumento tiende a infinito. En el contexto de algoritmos, describe una cota superior para el tiempo de ejecución (o espacio en memoria) en función del tamaño de la entrada. Por ejemplo, decir que el entrenamiento es $O(k \cdot w)$ implica que el tiempo de cálculo crece de manera proporcional al producto del número de datos k y el número de pesos w .

Definición 11 (Época de Entrenamiento): Una época (*epoch*) se define como un ciclo completo en el que el algoritmo de aprendizaje ha procesado la totalidad del conjunto de datos de entrenamiento una vez. Dado que el descenso por gradiente es iterativo, el entrenamiento de la red usualmente requiere múltiples épocas para que la función de costo converja a un mínimo aceptable.

9 Instrumentación y estrategias aplicadas

9.1 Entorno de hardware

Para la realización de las pruebas experimentales utilizamos dos equipos de cómputo con propósitos distintos. El primero, destinado al desarrollo y pruebas preliminares, estuvo disponible 24/7. El segundo, un equipo de alto rendimiento, se utilizó para la resolución masiva de los problemas de entrenamiento, aprovechando su tarjeta gráfica dedicada NVIDIA RTX 3090 Ti.

9.2 Implementación de software

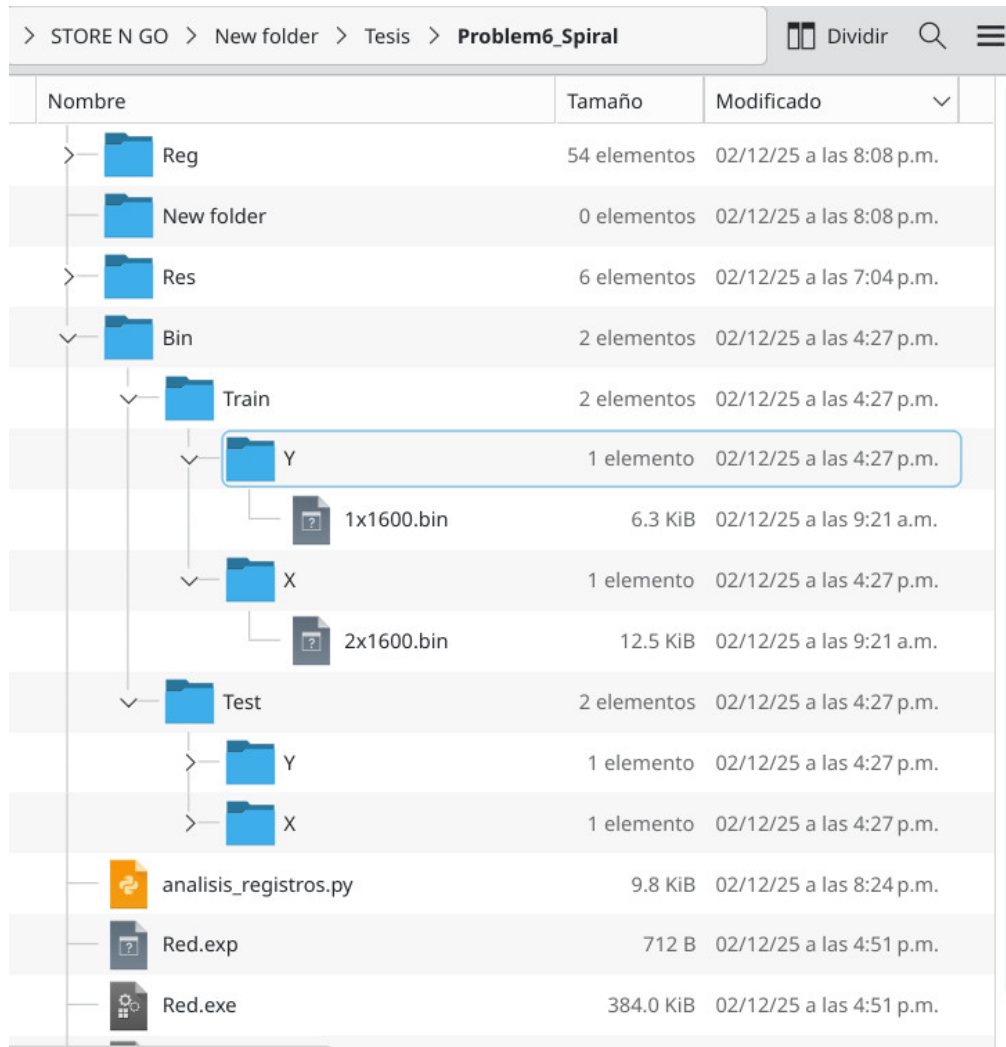
El desarrollo del software se dividió en dos componentes principales para aprovechar las fortalezas de distintos lenguajes:

1. Preprocesamiento y Prototipado (Python): El primer programa utiliza la librería *Pandas* para la lectura y curación (limpieza) de las bases de datos, transfiriendo posteriormente la información al componente encargado de la ejecución de las redes neuronales, el cual está escrito íntegramente utilizando la librería *NumPy*. Esta metodología resultó sumamente práctica para la fase de experimentación, ya que *Pandas* permite manipular bases de datos y realizar operaciones matriciales complejas con una sintaxis sencilla y eficiente.

2. Motor de Entrenamiento de Alto Rendimiento (C++/CUDA): El segundo programa fue escrito directamente en C/C++ utilizando la arquitectura CUDA para explotar el paralelismo de la GPU. En una etapa inicial intentamos implementar rutinas de lectura de archivos ‘.csv’ similares a las del programa en Python; sin embargo, este enfoque resultó ser innecesariamente complejo y tedioso en C++. Finalmente, optamos por una estrategia híbrida: el programa en CUDA recibe las bases de datos ya preprocesadas y estandarizadas por un script auxiliar más sencillo en Python.

9.3 Estrategia de gestión de archivos

Esta decisión metodológica nos llevó a mantener un orden estricto y estandarizado en la estructura de directorios y archivos donde se almacenan los modelos y datos. Esta rigidez estructural, lejos de ser un inconveniente, facilitó enormemente la recopilación de resultados y abrió la posibilidad de automatizar el preprocesamiento y el post-procesamiento con herramientas externas.



Nombre	Tamaño	Modificado
Reg	54 elementos	02/12/25 a las 8:08 p.m.
New folder	0 elementos	02/12/25 a las 8:08 p.m.
Res	6 elementos	02/12/25 a las 7:04 p.m.
Bin	2 elementos	02/12/25 a las 4:27 p.m.
Train	2 elementos	02/12/25 a las 4:27 p.m.
Y	1 elemento	02/12/25 a las 4:27 p.m.
1x1600.bin	6.3 KiB	02/12/25 a las 9:21 a.m.
X	1 elemento	02/12/25 a las 4:27 p.m.
2x1600.bin	12.5 KiB	02/12/25 a las 9:21 a.m.
Test	2 elementos	02/12/25 a las 4:27 p.m.
Y	1 elemento	02/12/25 a las 4:27 p.m.
X	1 elemento	02/12/25 a las 4:27 p.m.
analisis_registros.py	9.8 KiB	02/12/25 a las 8:24 p.m.
Red.exp	712 B	02/12/25 a las 4:51 p.m.
Red.exe	384.0 KiB	02/12/25 a las 4:51 p.m.

Figure 2: Estructura de archivos estandarizada para el problema 6 "Spiral".

Para ejecutar los experimentos, el binario compilado en CUDA se copia al directorio raíz que contiene esta estructura de archivos. Una vez allí, el programa inicia secuencialmente múltiples procesos de entrenamiento, variando la cantidad de neuronas y capas según la configuración deseada hasta completar todas las peticiones.

Conforme finaliza el entrenamiento de cada modelo, el sistema genera registros únicos identificados mediante un código *hash*. Estos archivos almacenan toda la información relevante del entrenamiento en un formato simple pero legible, listo para ser interpretado por las herramientas de análisis. Esta arquitectura de guardado permite detener el proceso en cualquier momento y reanudarlo posteriormente sin perder los registros ya generados.

Finalmente, los registros son procesados por un script de Python que se ejecuta en la misma carpeta, el cual se encarga de generar las gráficas comparativas y exportar archivos de hoja de cálculo (Excel/CSV) con los datos crudos y sus estadísticos más relevantes.

9.4 Obtención de los Problemas de Clasificación

Los problemas de clasificación binaria utilizados en este estudio fueron adquiridos mediante la librería *sklearn.datasets*. Esta herramienta nos permitió tanto generar conjuntos de datos sintéticos controlados como descargar bases de datos estandarizadas de repositorios públicos.

Linea	Registro (Red Angosta)	Linea	Registro (Red Profunda)
1		1	
2	Arquitectura: _2_6_1_	2	Arquitectura: _2_3_3_1_
3	Semilla: 1764697569	3	Semilla: 1764691097
4	Segundos: 108	4	Segundos: 351
5	Iteraciones: 500000	5	Iteraciones: 1000000
6		6	
7	Costo_train: 0.123278	7	Costo_train: 0.208237
8	Precision_train: 0.837532	8	Precision_train: 0.623377
9	Recall_train: 0.832290	9	Recall_train: 0.841051
10	F1_train: 0.834903	10	F1_train: 0.716036
11		11	
12	Costo_test: 0.123665	12	Costo_test: 0.209376
13	Precision_test: 0.815000	13	Precision_test: 0.617544
14	Recall_test: 0.810945	14	Recall_test: 0.875622
15	F1_test: 0.812968	15	F1_test: 0.724280
16		16	
17	Paso: 1.000000	17	Paso: 0.010000
18	Gamma: 0.300000	18	Gamma: 0.900000
19	Parada: 0.100000	19	Parada: 0.100000
20	Iteraciones: 500000	20	Iteraciones: 1000000
21	CapasDesde: 3	21	CapasDesde: 2
22	CapasHasta: 4	22	CapasHasta: 3
23	Repeticiones: 5	23	Repeticiones: 5
24	Intervalo: 10000	24	Intervalo: 10000
25	MultiplicadorProfunda: 0.700000	25	MultiplicadorProfunda: 0.700000
26	Tarjeta: NVIDIA GeForce GTX 950M	26	
27		27	Costos:
28	Costos:	28	0.3668880.3277400.3270870.3256200

Figure 3: Comparativa de archivos de registro: a la izquierda una red angosta y a la derecha su equivalente en una red profunda.

10 Resultados y Conclusiones

A continuación se presentan los resultados obtenidos para cada uno de los problemas de clasificación seleccionados, comparando el desempeño de las arquitecturas angosta y profunda bajo las condiciones experimentales descritas.

10.1 Problema 1: Blobs

- **Tipo:** Sintético (`make_blobs`).
- **Descripción:** Nubes de puntos separadas claramente.
- **Reto:** Suele ser linealmente separable. Se utiliza como control base.

Resultados: Todas las redes probadas resolvieron el problema sin dificultad, alcanzando un *F1-Score* del 100%, validando el correcto funcionamiento del algoritmo de entrenamiento.

10.2 Problema 2: Cáncer de Mama (Wisconsin)

- **Tipo:** Dataset real (Médico).
- **Descripción:** Características computadas a partir de una imagen digitalizada de un aspirado con aguja fina de una masa mamaria. Clasifica entre *Maligno* y *Benigno*.
- **Reto:** Alta dimensionalidad (30 características numéricas). El desafío radica en evitar el sobreajuste y determinar la relevancia de las características.

Resultados: Con 62 neuronas, ambas arquitecturas resuelven el problema eficazmente, obteniendo una media cercana al 95%. Es notable que los puntajes de prueba resultaron ligeramente superiores a los de entrenamiento, indicando una excelente generalización.

10.3 Problema 3: XOR (O exclusivo)

- **Tipo:** Sintético (Lógico).
- **Descripción:** Cuatro grupos de puntos donde las esquinas diagonales pertenecen a la misma clase.
- **Reto:** Caso de estudio clásico de no linealidad. Imposible de resolver con una sola línea recta; requiere necesariamente capas ocultas.

Resultados: Con tan solo 6 neuronas, ambas redes resuelven el problema con puntajes del 99.9%.

10.4 Problema 4: Círculos (Circles)

- **Tipo:** Sintético (`make_circles`).
- **Descripción:** Un círculo inscrito dentro de otro anillo concéntrico.
- **Reto:** Requiere una frontera de decisión radial cerrada.

Resultados: [Faltan resultados en el texto original].

10.5 Problema 5: Lunas (Moons)

- **Tipo:** Sintético (`make_moons`).
- **Descripción:** Dos semicírculos entrelazados.
- **Reto:** Evalúa la capacidad para modelar bordes curvos y no linealidades locales.

Resultados: Ambos modelos resuelven el problema con una puntuación aproximada del 89%. No se observa mejoría significativa al agregar más neuronas, sugiriendo que la complejidad del problema ha sido saturada.

10.6 Problema 6: Espiral (Spiral)

- **Tipo:** Sintético.
- **Descripción:** Dos o más brazos de espiral que parten del centro.
- **Reto:** Considerablemente más difícil que los anteriores. Requiere comprender la conectividad a lo largo de una trayectoria curva continua sin saltar entre brazos adyacentes.

Resultados: Este es el primer problema con dificultad suficiente para evidenciar diferencias estructurales entre las arquitecturas.

Con 6 neuronas, se obtiene una media de 72 puntos para ambas arquitecturas. Sin embargo, la desviación estándar de la red angosta es de 8 puntos, mientras que la profunda presenta mayor estabilidad con solo 4. No hay indicios de sobreajuste y ninguna red completó el entrenamiento antes del límite de 1 millón de épocas.

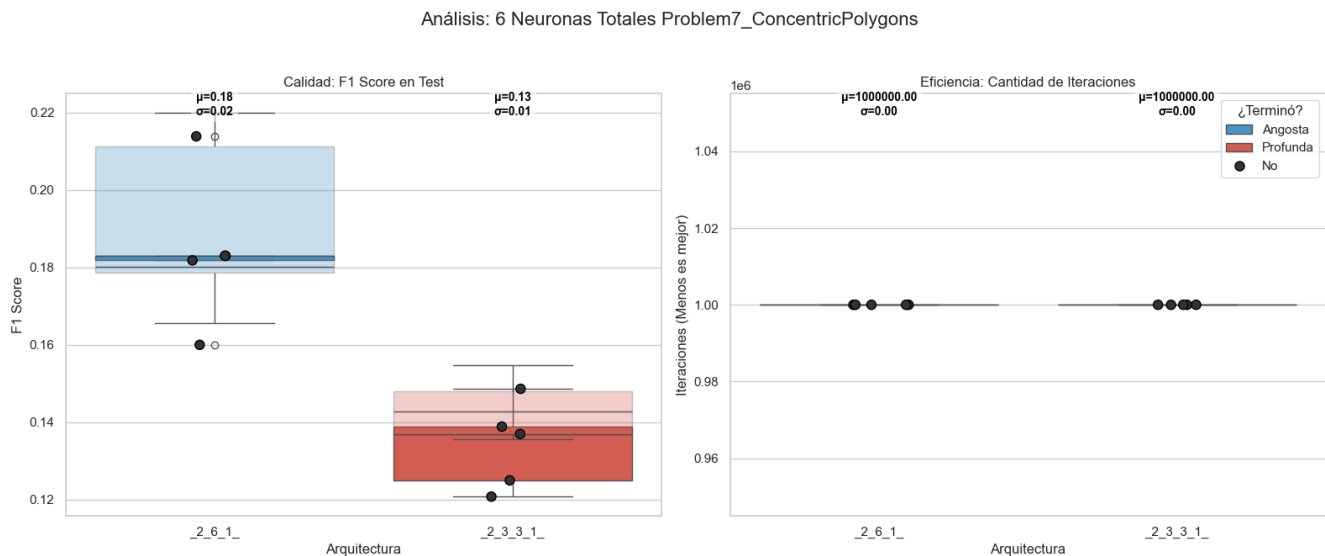


Figure 4: Comparativa de desempeño en el problema Espiral con 6 neuronas.

Con 9 neuronas, la red angosta mejora su puntaje al 85% con una desviación estándar reducida a solo 2 puntos. En contraste, la red profunda mantiene una media de 74 puntos pero con una varianza significativamente mayor (casi 9 puntos). Aunque existen casos aislados donde la red profunda logra resolver el problema, la red angosta lo hace en la totalidad de los casos y con menor número de iteraciones.

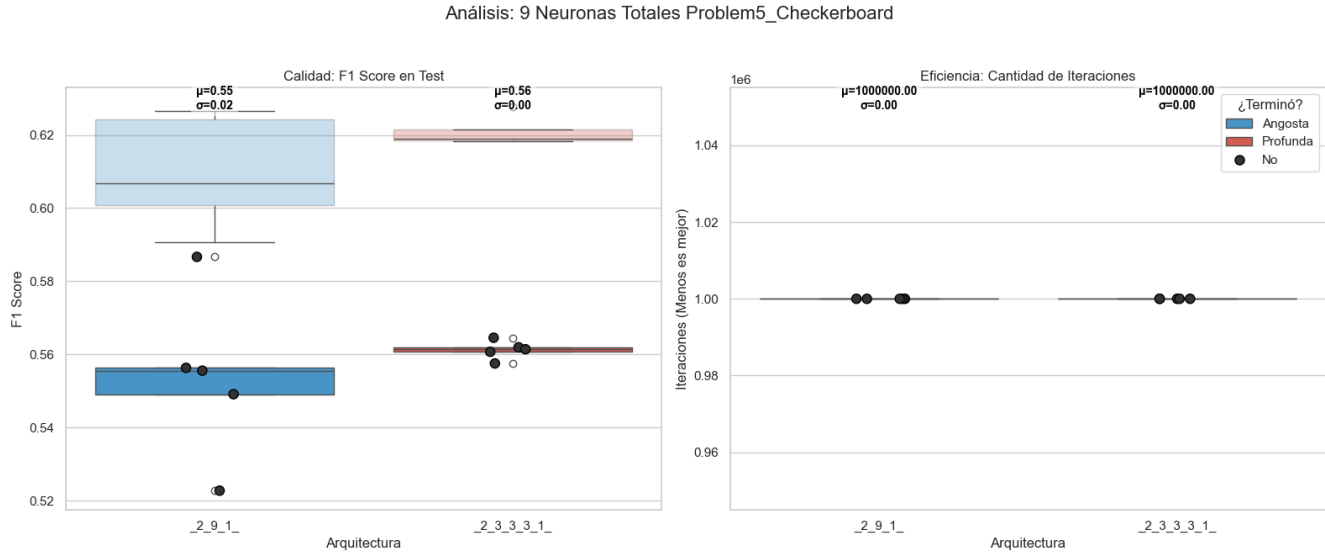


Figure 5: Comparativa de desempeño en el problema Espiral con 9 neuronas.

Con 12 neuronas, la superioridad de la red angosta es contundente, alcanzando un 89% de puntuación con baja desviación. La red profunda mejora marginalmente su media (2 puntos) pero a costa de una inestabilidad severa (desviación de 12 puntos); esto implica que mientras en algunos casos resuelve el problema en tiempos comparables a la angosta (200k épocas), en muchos otros diverge hacia resultados pobres cercanos al 60%.

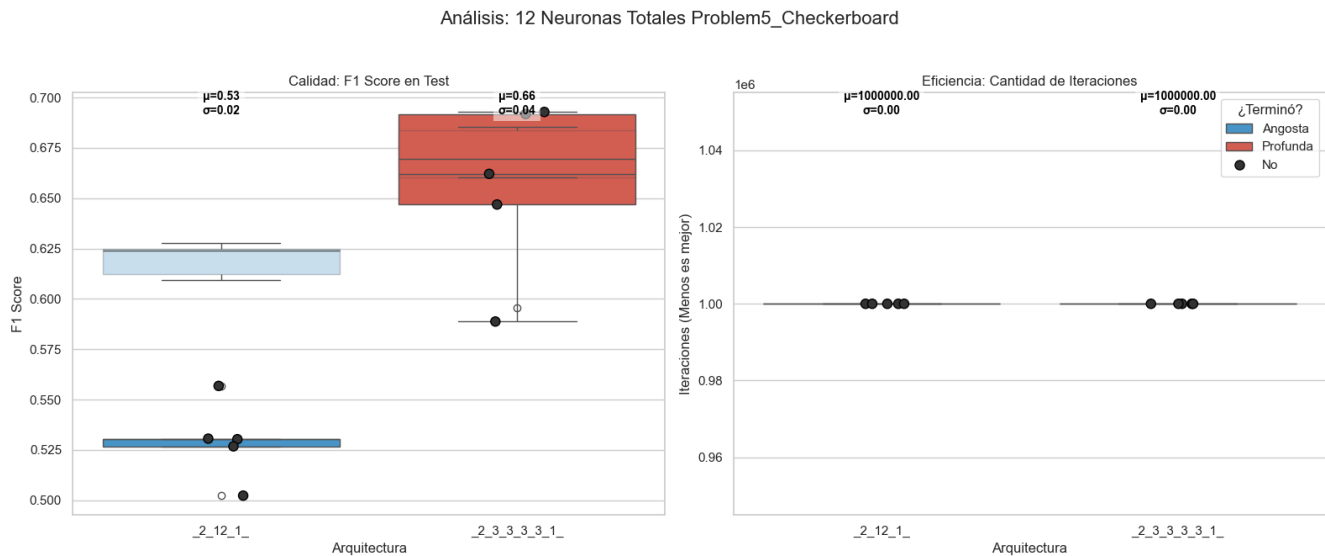


Figure 6: Comparativa de desempeño en el problema Espiral con 12 neuronas.

Concluimos que para este problema la arquitectura angosta es superior en consistencia y rendimiento, aunque la red profunda posee la capacidad teórica de resolverlo bajo condiciones ideales de inicialización.

10.7 Problema 7: Sonar (Minas vs. Rocas)

- **Tipo:** Dataset real (Señales/Física).
- **Descripción:** Datos de sonar rebotando contra un cilindro de metal o una roca.
- **Reto:** 60 características de frecuencia y pocos ejemplos (~ 208). El reto principal es la maldición de la dimensionalidad y el riesgo extremo de sobreajuste.

Resultados: Se probaron modelos de hasta 6 capas (366 neuronas equivalentes) sin lograr resolver el problema satisfactoriamente, probablemente debido a la escasez de datos para tal dimensionalidad.

10.8 Problema 8: Círculos Anidados (Nested Circles)

- **Descripción:** Un círculo pequeño de una clase rodeado por un anillo más grande de la otra clase.
- **Reto:** No linealmente separable, requiere fronteras curvas cerradas.

Resultados: Nuevamente, la red angosta presenta mejores resultados, con una media de puntuación superior y menor varianza que su contraparte profunda. Se observa una mejoría consistente, pasando del 89% al 95% al alcanzar las 12 neuronas. La red profunda también muestra progreso, aunque más lento.

10.9 Problema 9: Clústeres Sinusoidales (Sine Clusters)

- **Descripción:** Las clases siguen patrones de ondas entrelazadas.
- **Reto:** Frontera de decisión ondulada y periódica.

Resultados: Con 6 neuronas, ninguna arquitectura logra resolver el problema, quedando estancadas en una puntuación cercana al 50% (azar).

Al aumentar a 9 neuronas, la red angosta muestra una enorme mejoría, alcanzando consistentemente un puntaje de 85, mientras que la profunda se rezaga en 65. Ninguna resolvió el problema completamente dentro del límite de 1 millón de épocas.

Con 12 neuronas, la red angosta ya es capaz de resolver el problema con un consistente 85% y desviación mínima. La red profunda, por su parte, no logra superar la barrera del 60%.

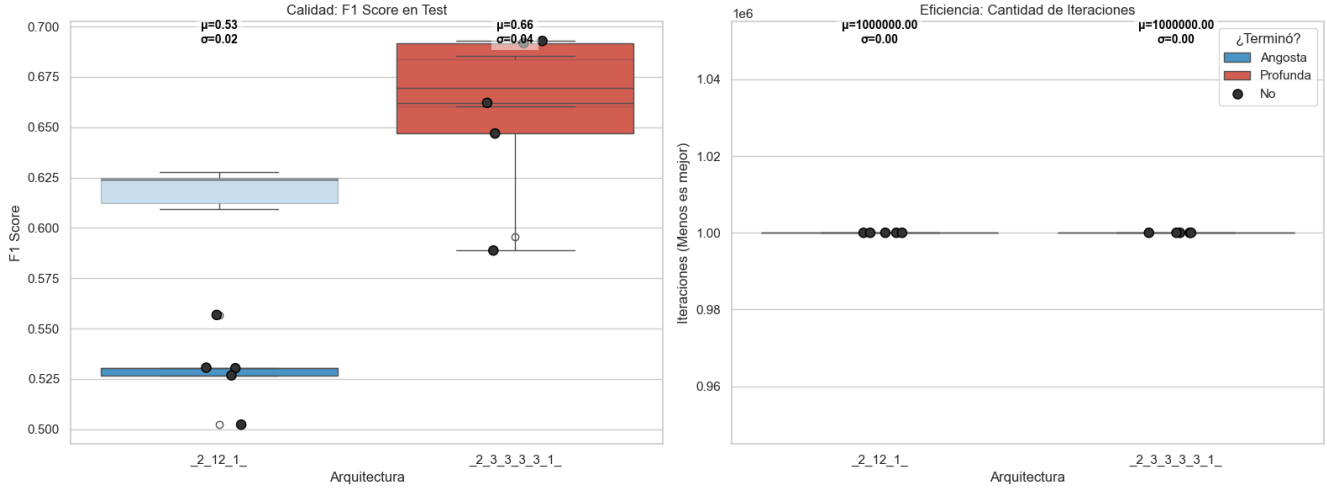


Figure 7: Comparativa en Círculos Anidados con 12 neuronas.

10.10 Problema 10: Malla Adaptativa (Adaptive Mesh)

- **Descripción:** Distribución de puntos con densidad variable.
- **Reto:** Adaptación de la frontera de decisión a zonas de distinta densidad.

Resultados: La red angosta resuelve este problema en tiempo récord y de forma consistente, mejorando con la adición de neuronas y manteniendo una desviación mínima (< 1). Por el contrario, la red profunda enfrenta dificultades crecientes al añadir complejidad; parece quedar atrapada en mínimos locales dependientes de la inicialización. Aunque teóricamente capaz, la probabilidad de convergencia exitosa disminuye con la profundidad.

10.11 Problema 11: Tablero de Ajedrez (Checkerboard)

- **Descripción:** Datos distribuidos en cuadrícula alternante.
- **Reto:** Generalización del XOR. Requiere crear regiones de decisión disconexas.

Resultados: Este es el primer caso donde se observa un sobreajuste claro, con una brecha de al menos 5 puntos entre entrenamiento y prueba para ambas arquitecturas. Con 6 y 9 neuronas, ninguna red resuelve el problema de manera satisfactoria.

Sin embargo, al llegar a 12 neuronas, ocurre un fenómeno interesante: la red profunda mejora su rendimiento hasta los 64 puntos y reduce significativamente los rastros de sobreajuste.

Finalmente, al agregar una cuarta capa (totalizando 15 neuronas equivalentes), la red profunda alcanza los 69 puntos sin varianza ni sobreajuste apreciable. La red angosta, en contraste, permanece estancada. Aunque el problema no se resolvió completamente, la tendencia sugiere que la

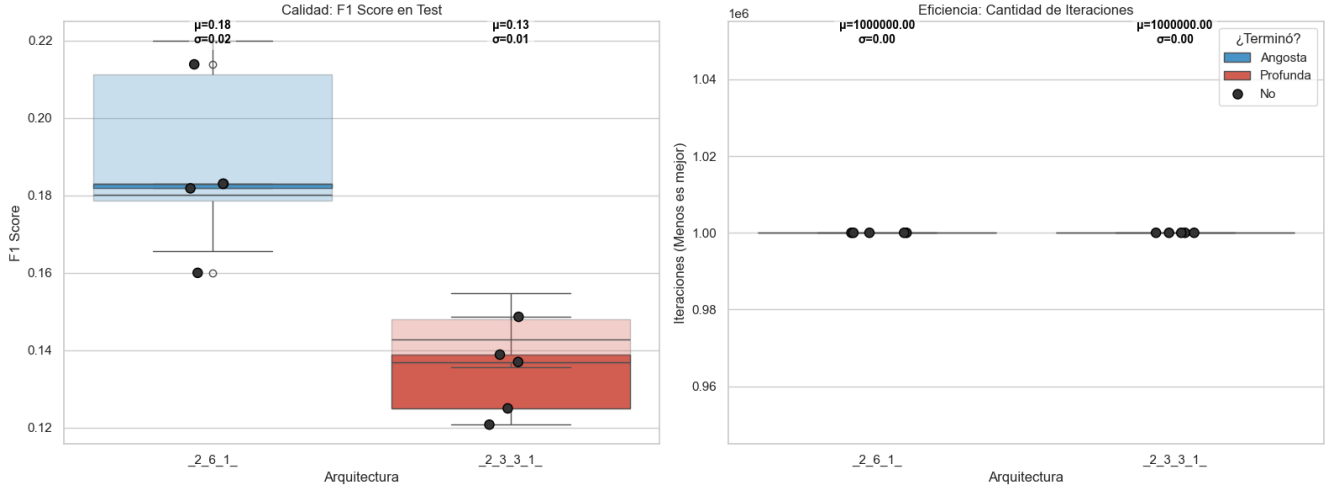


Figure 8: Desempeño en Clústeres Sinusoidales con 6 neuronas.

arquitectura profunda podría superarlo con mayor profundidad, siendo este el único caso donde mostró superioridad clara.

10.12 Problema 12: Rollo Suizo 2D (Swiss Roll)

- **Descripción:** Estructura en espiral densa.
- **Reto:** Aprendizaje de variedades (*manifold learning*). Requiere entender la continuidad estructural.

Resultados: Ninguna arquitectura resolvió el problema con pocas neuronas. La arquitectura profunda empeora su desempeño al agregar capas, mientras que la angosta mejora consistentemente pero sufriendo de un fuerte sobreajuste, estancándose finalmente en las 12 neuronas.

10.13 Problema 13: Polígonos Concéntricos

- **Descripción:** Formas geométricas de bordes rectos anidadas.
- **Reto:** Trazado de ángulos agudos y fronteras lineales precisas.

Resultados: Este experimento resultó irresoluble con el procedimiento actual, obteniendo puntuaciones muy bajas de 18 y 13 puntos para las redes angostas y profundas respectivamente, indicando una falla fundamental en la captura de la geometría del problema.

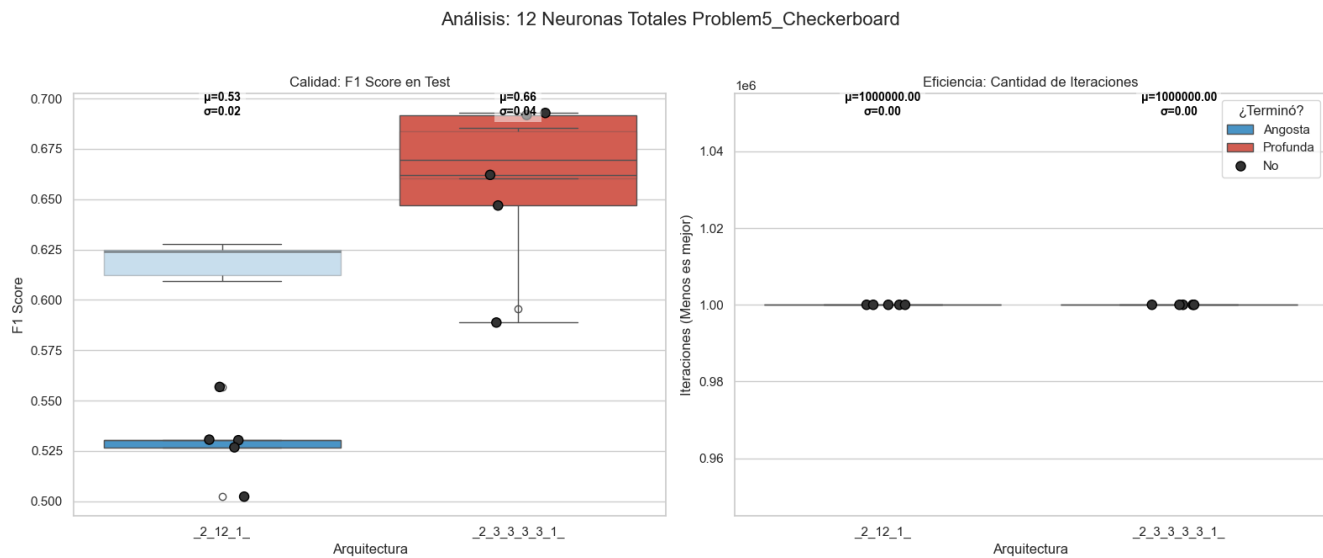


Figure 9: Desempeño en Clústeres Sinusoidales con 12 neuronas.

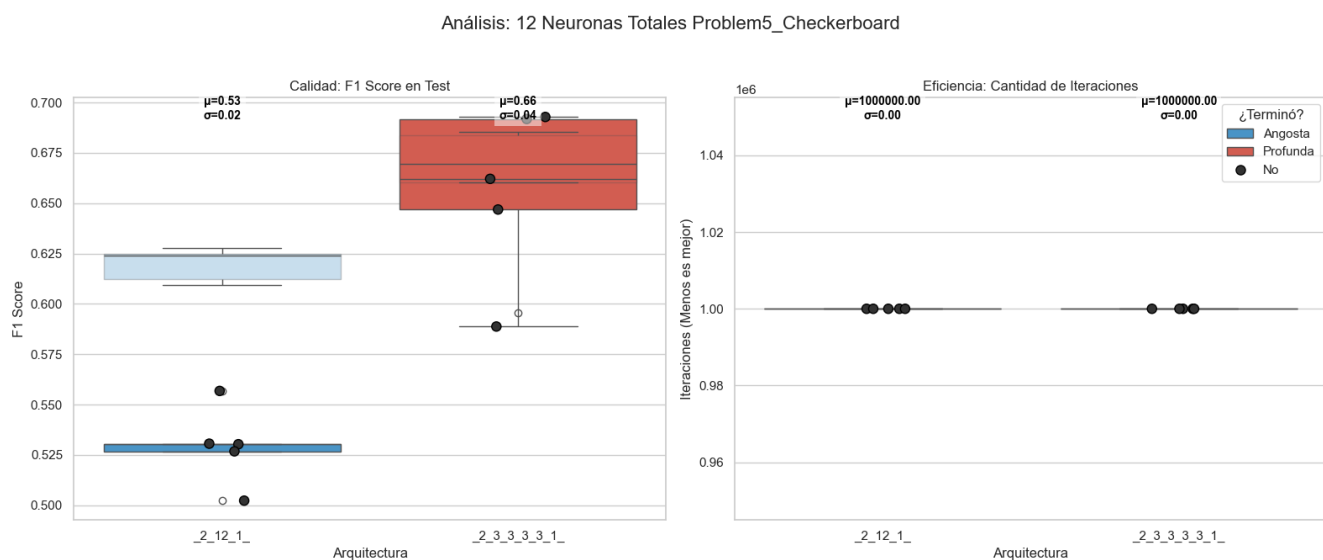


Figure 10: Desempeño en Malla Adaptativa.

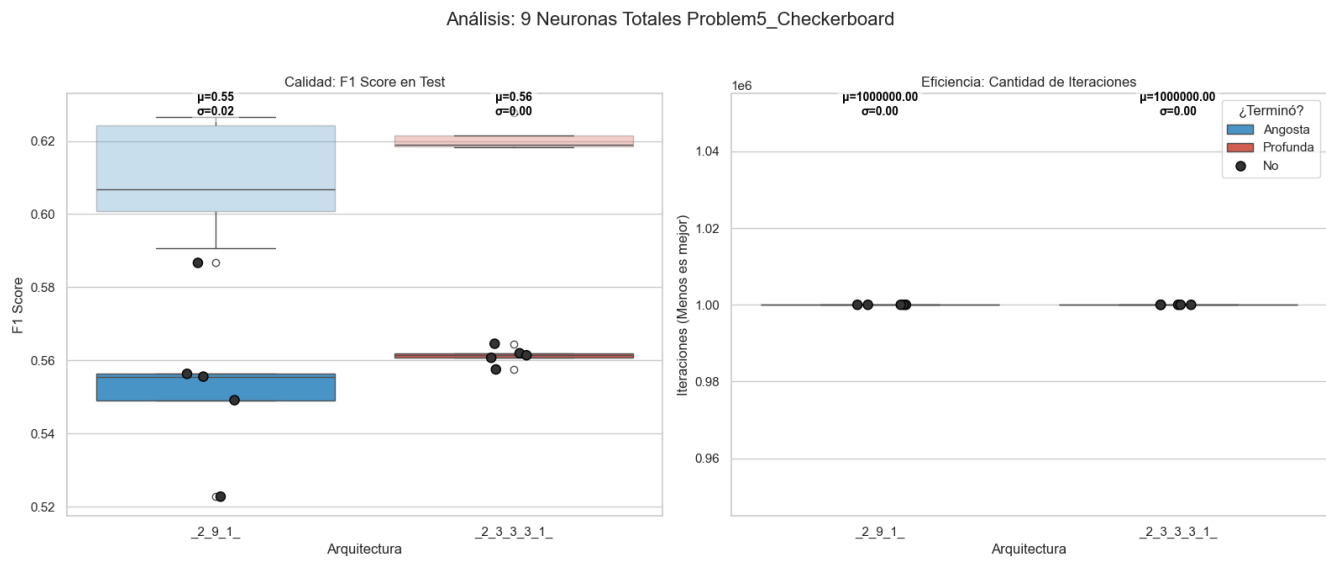


Figure 11: Desempeño en Tablero de Ajedrez con 9 neuronas.

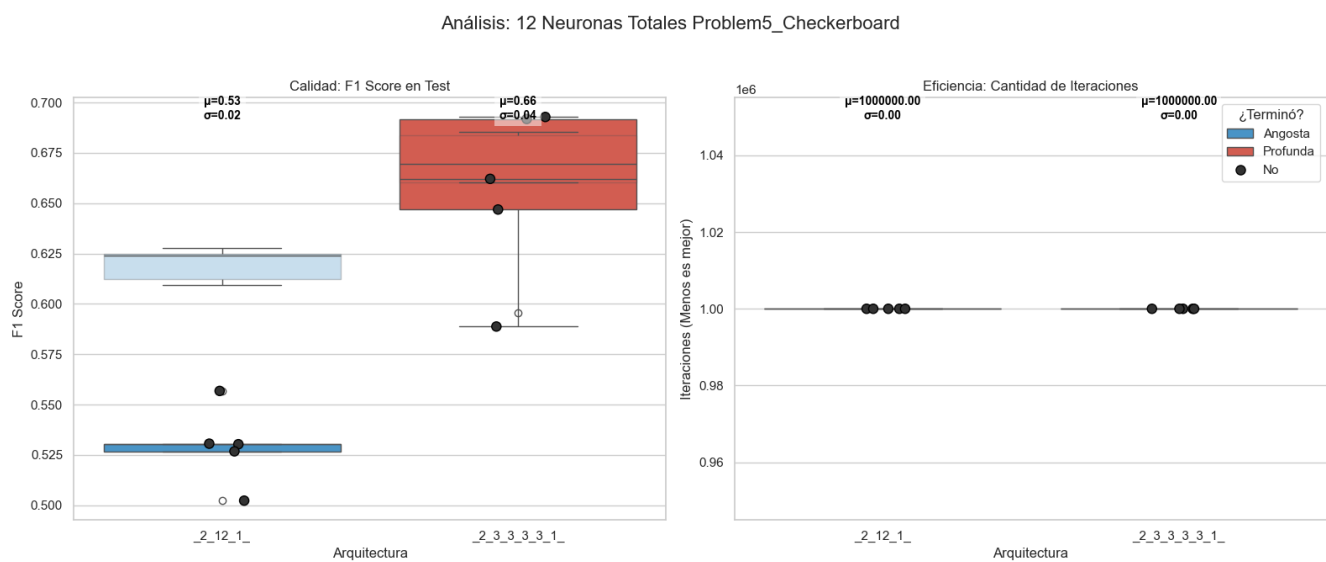


Figure 12: Desempeño en Tablero de Ajedrez con 12 neuronas.

Análisis: 15 Neuronas Totales Problem6_SwissRoll2D

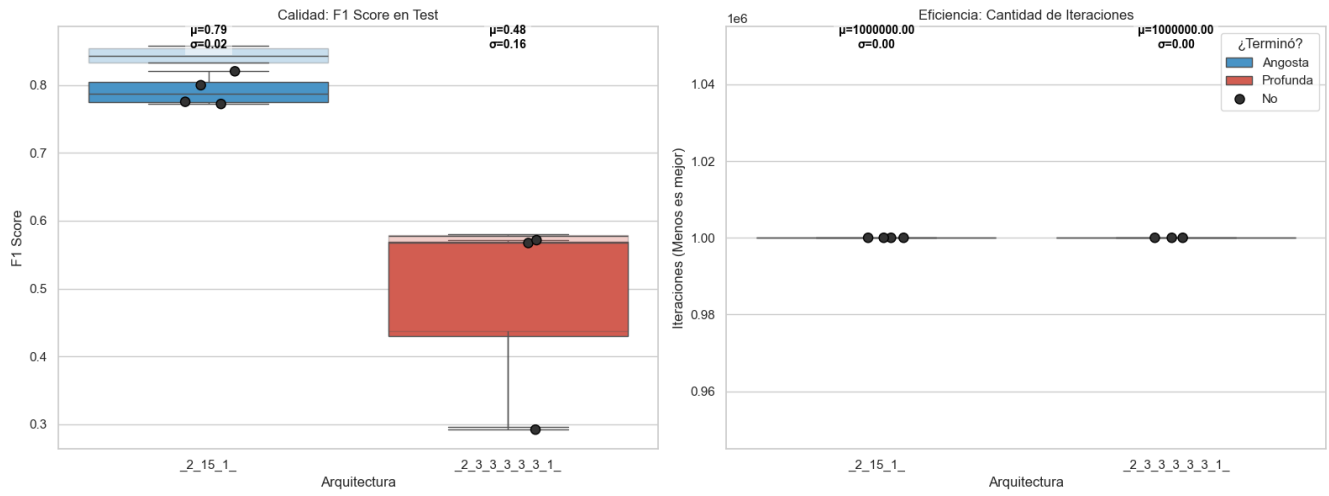


Figure 13: Desempeño con arquitectura extendida (15 neuronas).

Análisis: 15 Neuronas Totales Problem6_SwissRoll2D

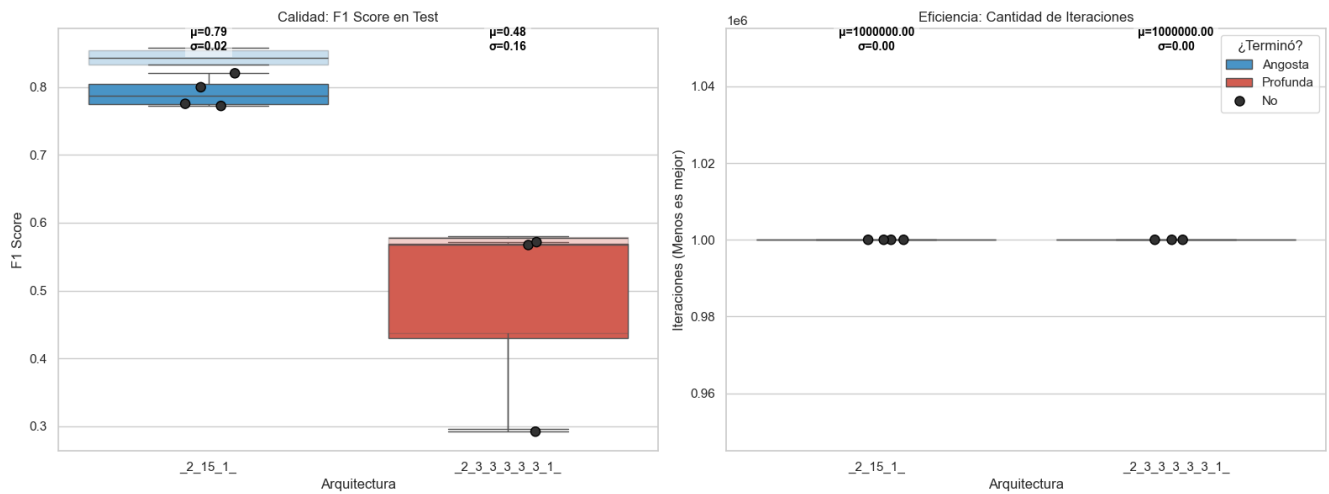


Figure 14: Resultados en Rollo Suizo 2D.

Análisis: 6 Neuronas Totales Problem7_ConcentricPolygons

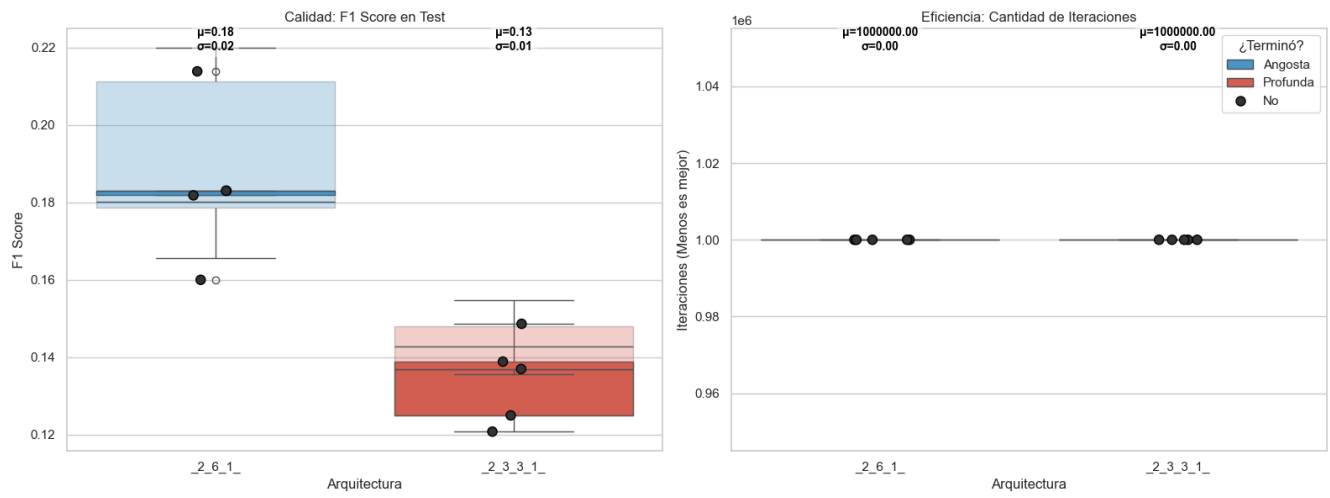


Figure 15: Desempeño deficiente en Polígonos Concéntricos.

11 Conclusiones Generales

Tras analizar los resultados experimentales, nos parece que para la mayoría de los problemas de clasificación de dificultad media que probamos, ambos enfoques funcionan, aunque con matices importantes. En general, la arquitectura **angosta** solía llevar un poco la ventaja, mostrando una convergencia más rápida y, sobre todo, más estable; cuando agregábamos neuronas, casi siempre veíamos una mejora, cosa que no siempre pasaba con las profundas.

En los casos que logramos resolver, las redes angostas solían dar mejores puntuaciones. Sin embargo, no podemos descartar a las redes profundas, ya que también lograron soluciones competitivas en varios escenarios, aunque notamos que a menudo les costaba más trabajo estabilizarse durante el entrenamiento.

Algo que resultó muy interesante de observar fue el patrón de mejora: en las redes angostas, el progreso es bastante predecible conforme se agrega ancho a la capa. Esto nos hace preguntarnos si, analizando estas primeras mejoras, sería posible saber de antemano en qué dirección nos conviene más hacer crecer la red (si a lo ancho o a lo profundo) sin tener que probarlo todo a ciegas.

La excepción a esta tendencia fue el problema del *Tablero de Ajedrez*. Aquí vimos que la red angosta se estancaba, mientras que la arquitectura profunda lograba reducir el sobreajuste y mejorar el rendimiento al agregar capas. Esto parece indicar que, al final del día, es la naturaleza misma del problema la que dicta la herramienta adecuada. Al final de cuentas por eso tenemos tarjetas de video y procesadores monolíticos, cada uno a su fin.

12 Bibliografia

- Ba, Jimmy and Rich Caruana (2014). “Do Deep Nets Really Need to be Deep?” In: *Advances in Neural Information Processing Systems*.
- Chatterji, Niladri S. and Philip M. Long (2023). “Deep Linear Networks can Benignly Overfit when Shallow Ones Do Not”. In: *Journal of Machine Learning Research* 24.188, pp. 1–26.
- Cybenko, G. (1989). “Approximation by superpositions of a sigmoidal function”. In: *Mathematics of Control, Signals, and Systems* 2.4, pp. 303–314. DOI: 10.1007/BF02551274.
- Fan, Simin (2024). “Deep Grokking: Would Deep Neural Networks Generalize Better?” In: arXiv: 2405.19454.
- Goodfellow, Ian, Yoshua Bengio, and Aaron Courville (2016). *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press.
- Kůrková, Věra (2020). “Limitations of Shallow Networks”. In: *Recent Trends in Learning From Data*. Springer. DOI: 10.1007/978-3-030-43883-8_6.
- Lu, Zhou et al. (2017). “The Expressive Power of Neural Networks: A View from the Width”. In: *Advances in Neural Information Processing Systems*.
- Mhaskar, Hrushikesh, Qianli Liao, and Tomaso Poggio (2017). “When and Why Are Deep Networks Better Than Shallow Ones?” In: *Proceedings of the AAAI Conference on Artificial Intelligence*. DOI: 10.1609/aaai.v31i1.10913.
- Nielsen, Michael A. (2015). *Neural Networks and Deep Learning*.
- Radhakrishnan, Adityanarayanan, Mikhail Belkin, and Caroline Uhler (2023). “Wide and deep neural networks achieve consistency for classification”. In: *Proceedings of the National Academy of Sciences* 120.15, e2208779120. DOI: 10.1073/pnas.2208779120.
- Yang, Yahong and Juncai He (2024). “Deeper or Wider: A Perspective from Optimal Generalization Error with Sobolev Loss”. In: arXiv: 2402.00152.